

Gemma 4 E4B QAT on Jetson Orin Nano 8GB

Complete Setup Guide for Local LLM HTML Game Generation

Platform: NVIDIA Jetson Orin Nano 8GB (Super), JetPack 6, CUDA 12.6, L4T R36.5.2. Model: Gemma 4 E4B QAT (Quantization-Aware Training, int4). Date: September 2026. All settings verified through systematic testing (21-config infrastructure sweep + 51-config sampling sweep).

1. Hardware

NVIDIA Jetson Orin Nano 8GB (Super) — 6-core Cortex-A78AE CPU, Ampere GPU (sm_87, 1024 CUDA cores, 32 tensor cores), 8GB unified LPDDR5 (CPU + GPU share one pool), ~102 GB/s memory bandwidth.

Power: USB-C 5V/4A (20W default) or barrel jack 9-20V (up to 25W MAXN Super mode). Solar setup: 100W panel -> Jackery Explorer 500 -> Jetson. Net solar output: 11.30W (100W x 19% panel efficiency x 70% real-world x 85% Jackery conversion).

2. Software Prerequisites

OS: Ubuntu 22.04 (JetPack 6, L4T R36.5.2). CUDA 12.6. Python 3.10+. Build tools: cmake, gcc/g++ 11+, git.

GPU clock: Run 'sudo jetson_clocks' to maximize GPU frequency before any inference. This gives +65% speed over default power mode.

GUI: Stop the display manager to free ~600MB RAM. Run: 'sudo systemctl stop gdm.service'. This is essential — the 8GB unified memory pool is shared between CPU and GPU, and GDM's overhead can cause OOM when loading 4GB+ models. To restart GUI later: 'sudo systemctl start gdm.service'.

3. Build llama.cpp with CUDA

Clone and build llama.cpp with CUDA support optimized for the Jetson's Ampere architecture:

```
git clone https://github.com/ggml-org/llama.cpp cd llama.cpp cmake -B build \ -DGGML_CUDA=ON \ -DCMAKE_CUDA_ARCHITECTURES="87" \ -DGGML_CUDA_F16=ON \ -DGGML_CUDA_FA=ON \ -DGGML_CUDA_FA_ALL_QUANTS=ON \ -DGGML_CUDA_GRAPHs=ON \ -DCMAKE_BUILD_TYPE=Release cmake --build build --config Release -j $(nproc)
```

Binaries at build/bin/llama-server, build/bin/llama-cli, build/bin/llama-bench.

Key build flags explained:

- GGML_CUDA=ON: Enable CUDA backend for GPU offload.
- CMAKE_CUDA_ARCHITECTURES=87: Target the Orin's Ampere SM 8.7 specifically.
- GGML_CUDA_F16=ON: Use FP16 tensor cores for matmul.
- GGML_CUDA_FA=ON: Flash attention with tensor cores.
- GGML_CUDA_FA_ALL_QUANTS=ON: Flash attention works with ALL KV cache quantization types. Without this, using flash attention with quantized KV silently falls back to CPU (25-45x slower, no warning).
- GGML_CUDA_GRAPHs=ON: CUDA graphs reduce kernel launch overhead.

4. Download the Model

The Gemma 4 E4B QAT model is a Quantization-Aware Training variant from Google/Unsloth. QAT models have pre-calculated static activations, reducing processing overhead on mobile/edge chips.

Download the Unsloth UD-Q4_K_XL quantization (4.0 GB):

```
wget -O ~/models/gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf \ "https://huggingface.co/unsloth/gemma-4-E4B-it-QAT-GGUF/resolve/main/gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf"
```

Model specs: Dense architecture, 42 layers, 2 KV heads, 256 head_dim, 18 shared KV layers (24 effective), sliding_window 512, 262K vocabulary. MatFormer: ~12B total parameters, 4B active per token. Multimodal (text + image + audio) but used text-only here. Supports function calling (tool_calls JSON).

5. Launch the Server

The complete launch command with all verified optimum settings:

```
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 \ ~/llama.cpp/build/bin/llama-server \ -m  
~/models/gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf \ --alias gemma-4-e4b-qat \ --host 127.0.0.1 --port  
8091 \ -ngl 99 -c 32768 \ -ctk f16 -ctv f16 \ -b 512 -ub 512 \ -fa on --jinja --fit off -np 1 -t 6 \ --temp 0.4  
--top-p 0.95 --top-k 40 --repeat-penalty 1.0
```

IMPORTANT: Use 'setsid' or subprocess.Popen(start_new_session=True) for launch. The terminal background wrapper adds ~200MB overhead, causing silent OOM on 8GB.

6. Infrastructure Settings (Verified)

Tested via 21-configuration sweep on September 25, 2026. Each setting tested independently with all others held at baseline.

Parameter	Value	Rationale
GGML_CUDA_ENABLE_UNIFIED_MEMORY	1	Runtime env var (not build flag). Switches cudaMalloc to cudaMallocManaged, letting the 4GB model use the full 8GB shared pool. Essential for >4GB models.
-ngl (GPU layers)	99	Offload all layers to GPU. On unified memory there is no transfer penalty.
-c (context)	32768	32K context. No speed penalty at any context size with f16 KV — the KV cache is small (~0.81GB at 32K).
-ctk / -ctv (KV cache)	f16 / f16	KEY FINDING: f16 is 4% faster than q4_0 for generation on Gemma 4's sliding-window architecture. q4_0 V-cache costs ~35% speed. Trade-off: f16 uses 0.81GB vs 0.40GB for q4_0 at 32K. Use q4_0 only if GUI must be on simultaneously.
-b / -ub (batch/ubatch)	512 / 512	7% faster prompt processing than 64/64 (the prior conservative setting). Does not affect generation speed (memory-bandwidth bound).
-fa (flash attention)	on	6x prompt processing speed if disabled. Always on. Requires GGML_CUDA_FA_ALL_QUANTS=ON at build time for quantized KV.
-t (threads)	6	No meaningful difference between 4/6/8 threads. Generation is GPU-bound, not CPU-bound.
--jinja	on	Required for proper chat template handling on Gemma 4 (thinking model). Without it, the model gets stuck in thinking loops.
--fit off	on	Required for llama.cpp builds after 0b1bad1. Prevents model fit errors.
--np (parallel slots)	1	Single slot only. Multiple slots cause OOM on 8GB.
load mode	auto (default)	mlock and mmap+mlock both hurt speed. mmap+mlock is 20% slower on prompt processing. Stick with auto.

7. Sampling Settings (Verified)

Tested via 51-test sweep (17 configs x 3 HTML project types) on September 25, 2026. Each variable tested independently. Three test types: Snake game (canvas game logic), Coffee landing page (responsive CSS), Data dashboard (tables + charts + filtering).

Parameter	Value	Rationale
temperature	0.4	0.4-0.6 sweet spot. Barely affects accuracy. Coffee and dashboard score 95/100 at all temps. Snake fluctuates 80-85.
top_k	40	Minimal effect across 20-64. 40 is safe default.
top_p	0.95	No measurable difference across 0.85-0.95.

Parameter	Value	Rationale
repeat_penalty	1.0	KEY FINDING: Disabling repeat penalty (1.0) gives +9% generation speed (17.5 vs 16.1 tok/s) with identical accuracy on 2/3 test types. HTML's repetitive structure benefits from not penalizing repetition. Only cost: Snake game drops from 85 to 75 (one failed check).
thinking	off (/no_think)	Saves ~10% tokens with no quality loss on HTML generation. Add 'no_think' to the system prompt. Thinking off actually scored higher on Snake (85 vs 80) and used fewer tokens (2180 vs 2451).

8. Performance Benchmarks

Raw benchmark (llama-bench, 1 rep, f16 KV, 8K ctx, batch 512):

Metric	Value
Prompt processing (pp512)	554 tokens/sec
Generation (tg128)	18.5 tokens/sec
Real-world CLI generation	17.9 tokens/sec
Model size on disk	4.0 GB (3.91 GiB)
Total parameters	7.46B (4B active per token)
RAM used (model + KV + overhead)	~5.1 GB
RAM free (GUI off)	~2.5 GB
Server startup time	~8 seconds

Comparison with prior model (Ornith-1.0-9B IQ3_M):

Metric	E4B QAT (new)	Ornith IQ3_M (prior)
Generation speed	17.5 tok/s	9.4 tok/s
Speed advantage	1.86x faster	baseline
Model size	4.0 GB	4.4 GB
Context window	32K	24K (48K truncated)
KV cache (32K)	0.81 GB (f16)	0.40 GB (q4_0)
HTML validation score	95-100/100	85-95/100
Function calling	Yes (tool_calls JSON)	No

9. OOM Prevention on 8GB

The 8GB unified memory pool is the primary constraint. These rules prevent OOM:

1. Stop GDM: 'sudo systemctl stop gdm.service' (frees ~600MB).
2. Use setsid to launch server, not terminal background wrapper (saves ~200MB overhead).
3. Restart server between EACH game to clear KV cache accumulation. Every 6 tests is not enough — the server crashes from KV buildup.
4. Drop caches between tests: 'sync && echo 3 | sudo tee /proc/sys/vm/drop_caches'.
5. After killing llama-server, GPU memory does NOT release cleanly — a full reboot may be needed to restart the server.
6. The /health endpoint returns 200 BEFORE the model is fully in VRAM — poll /v1/models instead.
7. Never run multiple model instances simultaneously on 8GB.
8. Delete stale response files before each curl request — a leftover file from a prior failed run will be parsed as an empty/error result.
9. Python urllib.request dies when the harness process is killed mid-generation, losing the response. Use curl as a separate subprocess writing to a file instead — curl survives independently and the output file persists even if the harness dies.

Memory budget at 32K context with f16 KV: - Model weights: 4.0 GB - KV cache (f16, 32K): 0.81 GB -
Compute buffers: ~0.15 GB - CUDA overhead: ~0.15 GB - Total: ~5.1 GB used, ~2.5 GB free (GUI off)

10. Sending Generation Requests

The server exposes an OpenAI-compatible API at `http://127.0.0.1:8091/v1/chat/completions`. Example using curl:

```
curl -s http://127.0.0.1:8091/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{ "model": "gemma-4-e4b-qat", "messages": [ {"role": "system", "content": "You are a web developer. Output only HTML. /no_think"}, {"role": "user", "content": "Create a Snake game in HTML5..."} ], "temperature": 0.4, "top_k": 40, "top_p": 0.95, "repeat_penalty": 1.0, "max_tokens": 20000, "stream": false }'
```

Key points: - Set `max_tokens` high (17000-20000) for full game generation. E4B typically uses 5000-10000 tokens and stops naturally. - Add `/no_think` to the system prompt to disable thinking mode (saves tokens, no quality loss). - Use `stream=false` for single-shot generation. Use `stream=true` for real-time output monitoring. - The response includes both `'content'` (the HTML) and `'reasoning_content'` (thinking, if enabled). - Set a long HTTP timeout (600-7200 seconds depending on expected output length).

10b. Hardened Test Harness Architecture

When running batch tests (multiple games back-to-back), the test harness must survive long generation times (5-10 minutes per game). Three approaches tested:

APPROACH 1 (FAILED): Python `urllib.request` in a single long-running process. The HTTP connection holds open for the entire generation. If the process is killed (OOM, gateway timeout, process cleanup), the connection breaks and the response is lost. The server keeps generating into the void, wasting GPU time and energy.

APPROACH 2 (BETTER): Python `subprocess` calling curl with `-o output_file`. Curl runs as a separate process writing the response to a file. If the Python harness dies, curl continues independently and the output file persists. However, the Python harness still needs to stay alive to parse the file afterward.

APPROACH 3 (RECOMMENDED): Bash script as the main process. Bash launches the server via `setsid` (fully detached), then calls curl directly (bash blocks until curl finishes). When curl returns, bash calls a short Python script to parse the response file (takes seconds, not minutes). This separates the long-running parts (server + curl) from the short-running part (parsing).

Key implementation details: - Delete any stale response file before starting curl: `'rm -f response_GAME.json'` - Server restart between each game: `'pkill -9 -f llama-server; sleep 3'` - Poll `/v1/models` (not `/health`) to confirm server readiness - Resume support: load existing results, skip games with `completion_tokens > 0` and no error key - Cache drop needs sudo: `'sync && echo 3 | sudo tee /proc/sys/vm/drop_caches'`

Files in `~/projects/e4b-solar-games-comparison/`: - `run_remaining_games.sh` — bash runner (recommended) - `run_game_v2.py` — Python harness with curl subprocess (fallback) - `parse_response.py` — short Python script to parse curl output - `gen_requests.py` — generates request JSON files from `test_config.json`

11. HTML Game Generation Tips

Findings from 19+ game prompts tested through both Ornith and E4B:

1. Keep prompts under 2,500 characters. Prompts over 2.5K chars are the #1 cause of truncation. Every fix that cut prompt below 2.5K succeeded. 2. Simplify core mechanics for complex genres. Defense games: single shield > multi-quadrant. Fighting: 3-state AI > 6-state. Simplifying eliminates repetition loops. 3. Lower temperature to 0.2 for strategy/fighting genres if repetition occurs. The default 0.4 works for most genres. 4. All graphics must be canvas-drawn — no external images, no CDNs, no libraries. All sounds via Web Audio API. 5. System prompt should specify: output complete HTML from to , no truncation, no placeholders, all JS in one script block.

Five failure modes identified (all preventable): - Mode 1: Oversized prompt (>2.5K chars). Fix: trim prompt. - Mode 2: Thinking repetition loop. Fix: shorter prompt + /no_think. - Mode 3: Genre complexity. Fix: simplify mechanics. - Mode 4: Code repetition loop. Fix: simplify CPU AI + lower temp. - Mode 5: Context exhaustion. Fix: reduce interconnected systems.

12. Project Locations

E4B optimization (infrastructure): ~/projects/e4b-optimization/ E4B optimum settings (sampling):
~/projects/e4b-optimum-settings/ E4B vs Ornith game comparison:
~/projects/e4b-solar-games-comparison/ E4B Serena agent settings: ~/projects/e4b-serena-settings/
Game prompt collection: ~/projects/ornith-q4km-retest/test_config.json (19 game prompts) Solar games
facts report: ~/projects/solar-games-report/solar-games-facts-report.pdf Jetson llama.cpp benchmarks:
~/projects/jetson-llamacpp-benchmarks/ LLM transmission router:
~/projects/llm-transmission-8gb-202609/

All E4B project start_agent.sh files use the same permanent optimized settings. The
e4b-serena-settings project provides a universal agent bridge (agent_bridge.py + agent_config.yaml)
for new projects: ./new_project.sh ~/projects/my-game 'prompt' then ./start_agent.sh.

Quick Reference Card

1. sudo systemctl stop gdm.service # free 600MB RAM 2. sudo jetson_clocks # max GPU clock 3.
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 \ ~/llama.cpp/build/bin/llama-server \ -m
~/models/gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf \ --port 8091 -ngl 99 -c 32768 \ -ctk f16 -ctv f16 -b
512 -ub 512 \ -fa on --jinja --fit off -np 1 -t 6 \ --temp 0.4 --top-p 0.95 --top-k 40 --repeat-penalty 1.0 4.
Wait ~8s for server 5. curl http://127.0.0.1:8091/v1/chat/completions with your prompt 6. /no_think in
system prompt for HTML generation 7. max_tokens=20000 for full game generation 8. Result: 17.5
tok/s, 95-100% accuracy, ~5.5 min per game